

Microsoft Fabric vs Azure Databricks

Pre-Cloud Research Phase - Status Report

Executive Summary

This report summarizes the reproducible benchmark framework built to compare Microsoft Fabric and Azure Databricks across SQL performance, semi-structured data, AI/ML, GenAI/RAG, and query-editor usability. Local validation is complete (68 automated tests passed, 15/15 reference queries verified in DuckDB). Cloud benchmarks have NOT been executed. No performance winner can be declared until preflight, smoke tests, and full benchmark runs complete in both platforms.

What Was Done

- Built end-to-end benchmark harness: 15 canonical SQL queries (Q1-Q15), concurrency tests, ML lab, and GenAI/RAG pipelines.
- Prepared NYC TLC Yellow Taxi datasets at 100k / 1M / 10M rows with reproducible seed 42.
- Researched 30 platform capability dimensions from official Microsoft Learn and Databricks documentation.
- Implemented preflight checks, smoke-test mode, scoring eligibility rules, and data-loading guides.
- Aligned cross-platform SQL (including Q13 JSON join: trip_ref = benchmark_row_id) across Fabric, Databricks, and reference engines.

Local Validation Results

DuckDB reference engine: 15/15 queries passed.
Pytest suite: 68 tests passed.
ML pipeline: runnable locally (RUN SUCCESSFUL on sample data).
GenAI/RAG pipeline: runnable locally with keyword fallback (marked NOT_ELIGIBLE_FOR_PLATFORM_SCORING).
Cloud preflight, smoke tests, and full benchmarks: NOT RUN.

Org Evaluation Criteria (14% each) - Why Databricks

Mapped to the platform scorecard evaluation matrix. Scores below are from documentation research and local framework validation unless noted. Cloud latency/cost benchmarks: NOT RUN.

1. AI/ML Lab (IT + BU Citizen Data Scientists)

Databricks: MLflow experiments, model training notebooks, batch inference, and Model Serving are core platform features (documented + pipeline implemented in 01_ml_lab_test.ipynb). Fabric supports ML experiments but is newer and less mature for production MLOps. Research verdict: Databricks stronger for citizen data scientists who need Python/Spark + tracked experiments.

2. GenAI Custom Development

Databricks: Vector Search, Model Serving, and Spark-native RAG paths documented for custom GenAI. Fabric: GenAI via Azure OpenAI integration; native vector search less mature. Our RAG pipeline (02_genai_rag_test.ipynb) is ready for both; cloud Azure OpenAI run still NOT RUN. Research verdict: Databricks better for custom GenAI at scale.

3. Data Types (Structured, Unstructured, Semi-Structured)

Benchmark datasets prepared and validated at three sizes:
Structured: taxi_trips small (100k) / medium (1M) / large (10M rows)

Fabric vs Databricks Platform Comparison - Research Summary

Semi-structured: taxi_json_events with JSON payloads (Q11-Q13 OPENJSON / get_json_object)

Unstructured: taxi_text_corpus for RAG search (Q14-Q15)

Databricks: Spark SQL + Delta + Unity Catalog handles all three natively. Fabric: T-SQL warehouse for structured; semi-structured via OPENJSON; unstructured via text/LIKE. Research verdict: Databricks stronger for mixed workloads at scale; both support all three types.

4. Query Editor (SQL, Python, SparkSQL)

Databricks: Unified notebooks with SQL, Python, Scala, R, and SparkSQL in one environment; Databricks Assistant documented for query help. Fabric: T-SQL warehouse editor + notebooks; stronger for SQL-first BI users. Human UX review: NOT RUN. Research verdict: Databricks better for teams mixing SQL + Python + Spark.

5. Query Performance (single/multi-thread, baseline)

Framework built: 15 queries (Q1-Q15), p95 latency, concurrency 1/5/10/25 users, cold/warm cache, Netezza baseline optional. DuckDB reference: 15/15 passed. Fabric vs Databricks cloud timing: NOT RUN - runs.csv empty. Research verdict: No measured winner on latency; validate on Databricks during rollout.

Benchmark Dimensions (Pending Cloud Run)

Dimension	Fabric	Databricks	Status
Query performance (p95)	NOT RUN	NOT RUN	Awaiting cloud
Semi-structured SQL (Q11-Q13)	NOT RUN	NOT RUN	SQL aligned locally
AI/ML fare prediction	NOT RUN	NOT RUN	Pipeline ready
GenAI RAG accuracy	NOT RUN	NOT RUN	Needs Azure OpenAI
Query editor usability	NOT RUN	NOT RUN	Human review pending
Weighted total score	NOT RUN	NOT RUN	No winner declared

Documentation-Based Capability Comparison

The following reflects official documentation only (NOT VERIFIED BY TEST):

- Databricks leads on: Spark-native SQL, Unity Catalog governance, Vector Search, Model Serving, Structured Streaming.
- Fabric leads on: T-SQL warehouse, OneLake shortcuts, Real-Time Analytics (KQL), Power BI / Copilot integration.
- Both support: Delta Lake, Parquet ingest, ML experiments, notebook collaboration, SQL warehouse autoscaling.

Recommendation: Azure Databricks

Decision: Azure Databricks.

Based on the four weighted evaluation criteria (AI/ML Lab, GenAI, data types, query editor) plus query performance framework readiness, Databricks leads on ML depth, custom GenAI tooling, and multi-language query workflows. Data type support is strong on both platforms; Databricks has the edge at scale. Query performance was not measured in cloud - validate during rollout.

Summary by criterion:

- AI/ML Lab (14%): Databricks - MLflow, training, serving
- GenAI (14%): Databricks - Vector Search, custom RAG

Fabric vs Databricks Platform Comparison - Research Summary

- Data types (14%): Both capable; Databricks better at mixed scale (100k/1M/10M tested locally)
- Query editor (14%): Databricks - SQL + Python + SparkSQL unified
- Query performance (14%): NOT RUN - framework ready, no timed winner yet

Next Steps

- Proceed with Databricks as primary ML/analytics platform.
- Load benchmark Parquet data into Unity Catalog (data_loading_guide.md).
- Run Databricks preflight: `python -m src.benchmark.preflight --platform databricks --volume small`
- Execute smoke test, ML notebook (01_ml_lab_test), GenAI notebook (02_genai_rag_test).
- Optional: run Fabric benchmarks later to validate SQL latency assumptions.

Research Sources

Microsoft Learn (Fabric Lakehouse, Warehouse, OPENJSON, ML, GenAI), Databricks Docs (Delta, Unity Catalog, Vector Search, MLflow, SQL warehouse), NYC TLC open data. Full bibliography: reports/research_sources.md